

Lecture 9

Polymorphism, Part 1 Intro

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Break Time

`bit.ly/4ctTNbp`



Types of Polymorphism

1. Subtype Polymorphism
2. Ad hoc polymorphism
3. Parametric polymorphism

Subtype Polymorphism

```
export interface Notification {
  send(): void;
  format(): string;
}

export class EmailNotification implements Notification {
  constructor(
    private recipient: string, private subject: string,
    private message: string
  ) {}

  send(): void { /* Connect to SMTP server and send email */ }
  format(): string {
    return `To: ${this.recipient}\nSubject:
    ${this.subject}\n\n${this.message}`;
  }
}

export class SMSNotification implements Notification {
  constructor(private phoneNumber: string,
    private message: string) {}

  send(): void { /* Connect to SMS gateway and send text */ }
  format(): string {
    return `SMS to ${this.phoneNumber}: ${this.message}`;
  }
}
```

```
class NotificationService {
  // Can hold any type that implements Notification.
  private queue: Notification[] = [];

  enqueue(notification: Notification): void {
    this.queue.push(notification); }

  processAll(): void {
    for (const notification of this.queue) {
      console.log(`Sending: ${notification.format()}`);
      notification.send();
    }
    this.queue = [];
  }
}

function main() {
  const service = new NotificationService();
  service.enqueue(new EmailNotification("...", "...", "..."));
  service.enqueue(new SMSNotification("...", "..."));
  service.processAll();
}

main();
```

Subtype Polymorphism

- Depend only on a polymorphic Notification **base interface**
- There may be many **subtypes** that implement the interface
- The set of subtypes is **opaque**
 - We don't depend on specific subtypes when **using** them.
 - However, we do need to know the subtypes to **create** them.

Subtype Polymorphism Across Languages

- In some languages like Typescript, structural typing is pretty much all you need for subtype polymorphism.
- Other languages use a nominal type system and subtype polymorphism can only be achieved through extends, implements, etc.
- Example – in C++ use an abstract base class with pure virtual functions to define a base interface.

Subtype Polymorphism and “Inheritance”

```
export abstract class Notification {
  constructor(protected rec: string, protected msg: string) {}
  abstract send(): void;
  abstract format(): string;
}

export class EmailNotification extends Notification {
  constructor(rec: string, private sub: string,
    msg: string) { super(rec, msg); }

  send(): void { /* Connect to SMTP server and send email */ }
  format(): string {
    return `To: ${this.rec}\nSubject:
    ${this.sub}\n\n${this.msg}`;
  }
}

export class SMSNotification extends Notification {
  constructor(private phoneNumber: string, message: string) {
    super(phoneNumber, message);
  }

  send(): void { /* Connect to SMS gateway and send text */ }
  format(): string {
    return `SMS to ${this.phoneNumber}: ${this.message}`;
  }
}
```

```
class NotificationService {
  // Can hold any type that implements Notification.
  private queue: Notification[] = [];

  enqueue(notification: Notification): void {
    this.queue.push(notification);
  }

  processAll(): void {
    for (const notification of this.queue) {
      console.log(`Sending: ${notification.format()}`);
      notification.send();
    }
    this.queue = [];
  }
}

function main() {
  const service = new NotificationService();
  service.enqueue(new EmailNotification("...", "...", "..."));
  service.enqueue(new SMSNotification("...", "..."));
  service.processAll();
}
```

“Interface Inheritance” via implements
“Implementation Inheritance” via extends

Ad Hoc Polymorphism

```
export type NotificationKind = "email" | "sms";

export interface EmailData {
  kind: "email";
  recipient: string;
  subject: string;
  message: string;
}

export interface SMSData {
  kind: "sms";
  phoneNumber: string;
  message: string;
}

export type NotificationVariant = EmailData | SMSData;

// Helper for checking exhaustiveness
export function assertNever(x: never): never {
  throw new Error("Unexpected: " + x);
}
```

```
const service = new NotificationService();
service.enqueue({ kind: "email", rec: "...", sub: "...", msg: "..."});
service.enqueue({ kind: "sms", phoneNumber: "...", message: "..."});
service.processAll();
```

```
class NotificationService {
  private queue: NotificationVariant[] = [];

  enqueue(notification: NotificationVariant): void {
    this.queue.push(notification);
  }

  processAll(): void {
    for (const notification of this.queue) {
      console.log(this.format(notification));
      this.send(notification);
    }
    this.queue = [];
  }

  private send(notification: NotificationVariant): void {
    switch (notification.kind) {
      case "email": console.log(`Sending email to
${notification.recipient}...`); break;
      case "sms": console.log(`Sending SMS to
${notification.phoneNumber}...`); break;
      default: assertNever(notification);
    }
  }

  private format(notification: NotificationVariant): string {
    switch (notification.kind) {
      case "email": ...;
      case "sms": ...;
      default: assertNever(notification);
    }
  }
}
```

Ad Hoc Polymorphism

- Depend on a union type of all possible **variants** of Notification.
- Functions that want to operate on notifications are polymorphic and must specify behavior for each variant in an “ad hoc” or “one-by-one” way.
- The set of variants is known to the whole program.
 - We consider each of the variants when **using** them.
 - We can just create the appropriate variant/data to **create** them

Grid Class

- Consider making a 2D grid class...
- Can we do this with subtype polymorphism?

```
class UnsafeGrid {  
  private cells: any[][];  
  
  constructor(width: number, height: number, defaultValue: any) {  
    this.cells = Array.from({ length: height }, () =>  
      Array.from({ length: width }, () => defaultValue)  
    );  
  }  
  
  get(x: number, y: number): any {  
    return this.cells[y][x];  
  }  
  
  set(x: number, y: number, value: any): void {  
    this.cells[y][x] = value;  
  }  
}
```

Grid Class

- Consider making a 2D grid class...
- Can we do this with ad hoc polymorphism?

```
// We could try to enumerate every possible grid type as variants...
type GridKind = "mine" | "shield" | "string" | "number";

type GridVariant =
  | { kind: "mine"; cells: Mine[][] }
  | { kind: "shield"; cells: Shield[][] }
  | { kind: "string"; cells: string[][] }
  | { kind: "number"; cells: number[][] };

// But now we need separate getters that return different types!
function getFromMineGrid(grid: GridVariant & { kind: "mine" },
  x: number, y: number): Mine {
  return grid.cells[y][x];
}

function getFromShieldGrid(grid: GridVariant & { kind: "shield" },
  x: number, y: number): Shield {
  return grid.cells[y][x];
}

function getFromStringGrid(grid: GridVariant & { kind: "string" },
  x: number, y: number): string {
  return grid.cells[y][x];
}

// ... and so on for every type we might ever want to store!
```

Parametric Polymorphism

- Parametric polymorphism essentially allows unlimited variation across one or more type parameters.
- We'll come back to this more thoroughly in the future...

```
class SafeGrid<T> {  
  private cells: T[][];  
  
  constructor(width: number, height: number, defaultCell: () => T) {  
    this.cells = Array.from({ length: height }, () =>  
      Array.from({ length: width }, () => defaultCell())  
    );  
  }  
  
  at(x: number, y: number): T {  
    return this.cells[y][x];  
  }  
  
  set(x: number, y: number, value: T): void {  
    this.cells[y][x] = value;  
  }  
}
```